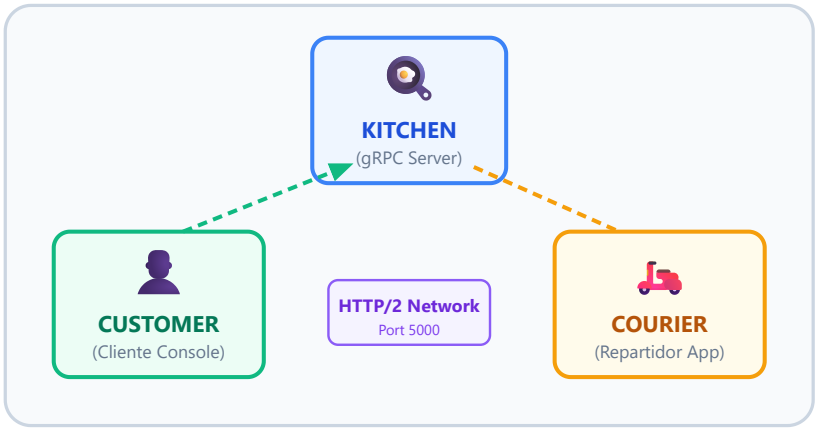
DISTRIBUTED SYSTEMS · PRACTICAL GUIDE

gRPC Made Simple

A Beginner's Walkthrough to Building a Distributed Restaurant System from Scratch



Based on Distributed Systems Lab 2 (UCLM)

Step-by-Step Practical Companion · C# & .NET 8

OVERVIEW

Table of Contents

Your roadmap to gRPC mastery

Sec	Topic	Page
1	Title & Table of Contents	1 – 2
2	The Big Picture & Restaurant Analogy	3 – 4
3	Why gRPC? The REST/JSON Problem	5
4	How Pieces Talk & RPC Flow	6 – 7
5	The .proto File Line by Line	8 – 9
6	Code Generation (protoc)	10
7	The Lab Step by Step	11 – 21
8	Common Mistakes & Fixes	22
9	Cheat Sheet & 5 Quiz Questions	23 – 24

💡 **Blue Box: Why?**
Explains engineering reasons behind choices.

📌 **Green Box: Remember**
Key rules to keep programs running smoothly.

⚠️ **Red Box: Warning**
Common stumbling blocks causing runtime errors.

📝 **Amber Box: Notes**
Clarifies missing or unclear lab details.

What We Build

A distributed restaurant across three independent programs

Welcome! We are building an online order system for **"Distributed Systems and Fun"**.

We build **three console programs**:

- **Restaurante (Kitchen)**: Central gRPC server with a thread-safe order queue.
- **Cliente (Customer)**: Places orders and queries status.
- **Repartidor (Courier)**: Waits for ready food and delivers it.

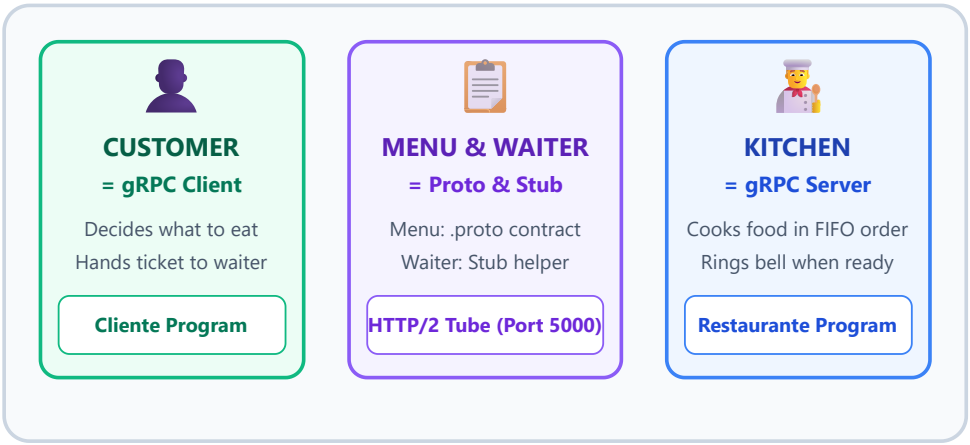
You will run all three in separate terminals and watch them interact in real time!

> 💡 **Why separate programs?**
> In real systems, customers, kitchens, and drivers run on different machines and must communicate reliably.

The Restaurant Analogy

Mapping real restaurant roles directly to gRPC components

To make distributed systems easy to understand, we use **one single analogy** throughout this guide:



> 📌 **Remember**
>
> Each program runs in its own console window without sharing memory. All data moves over HTTP/2 using binary Protocol Buffers.

The Problem It Solves

Moving beyond slow, verbose REST and JSON

Traditional APIs use **REST** with **JSON text**. While readable, JSON has big drawbacks:

- **Bulky Text**: Repeating keys like `"id_pedido"` wastes bandwidth.
- **Slow Parsing**: Parsing numbers from text burns CPU cycles.
- **No Strict Contract**: Typos cause silent runtime crashes!

Restaurant Analogy: REST is mailing a long letter to order soup. gRPC is sending a numbered ticket through a pneumatic tube: instant, tiny, and foolproof.

> 💡 **Why is gRPC faster?**
> gRPC encodes messages in binary Protocol Buffers: up to 7x smaller and 8x faster than JSON!

The Five Key Players

How client, server, stub, HTTP/2, and protobuf work together

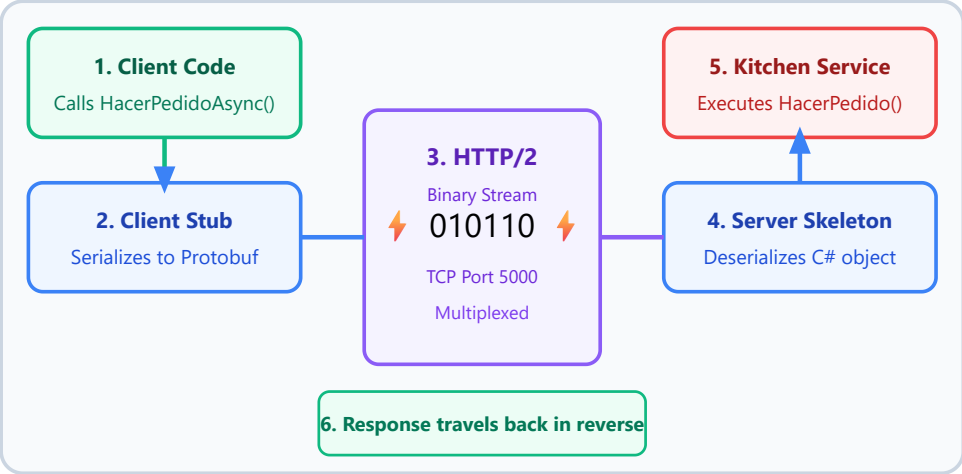
Meet the five components that make gRPC communication work seamlessly:

- **Client**: The app requesting work (the customer).
- **Server**: The central app doing the work (the kitchen).
- **Client Stub (Proxy)**: A generated local helper class that makes remote calls feel like local C# methods.
- **Protocol Buffers**: Google's compact binary format for encoding data.
- **HTTP/2**: Modern transport protocol that multiplexes many requests over one connection.

> 📌 **Remember: Network Transparency**
>
> Network transparency lets you write `await client.HacerPedidoAsync()` without writing raw network socket code!

Request & Response Flow

Tracing an RPC call from C# method to binary bytes and back



💡 Marshalling vs Unmarshalling

Marshalling packs a C# object into binary bytes for transmission.
Unmarshalling unpacks binary bytes back into a live C# object.

The .proto File: Messages

The agreement that client and server both honor

The `.proto` file uses Protocol Buffers syntax to define all exchanged messages:

```
syntax = "proto3";

message PlatoPedido {
  string nombre = 1;    // Field tag 1
  int32 cantidad = 2;   // Field tag 2
}
```

What do `= 1;` and `= 2;` mean?

These are **binary field tags**, NOT default values or array indexes!

In the binary packet, gRPC sends the single number `1` instead of the text `"nombre"`. This keeps messages tiny.

> ⚠️ **Warning: Never renumber fields!**
> Never swap tag numbers after releasing code. If tag 1 was a string, renumbering it causes servers to read corrupted data!

The .proto File: Services

Declaring the kitchen's public capabilities

A **service** is the menu of operations available on the server. Each operation is an **rpc**:

```
service RestauranteService {
  // 1. Customer places an order
  rpc HacerPedido (PedidoRequest) returns (PedidoResponse);

  // 2. Customer checks order status
  rpc ConsultarEstadoPedido (ConsultaRequest) returns (EstadoResponse)

  // 3. Courier waits and claims an order
  rpc TomarPedidoParaRepartir (google.protobuf.Empty) returns (PedidoR

  // 4. Customer inspects order dishes
  rpc ConsultarPlatosPedido (ConsultaRequest) returns (PedidoResponse)
}
```

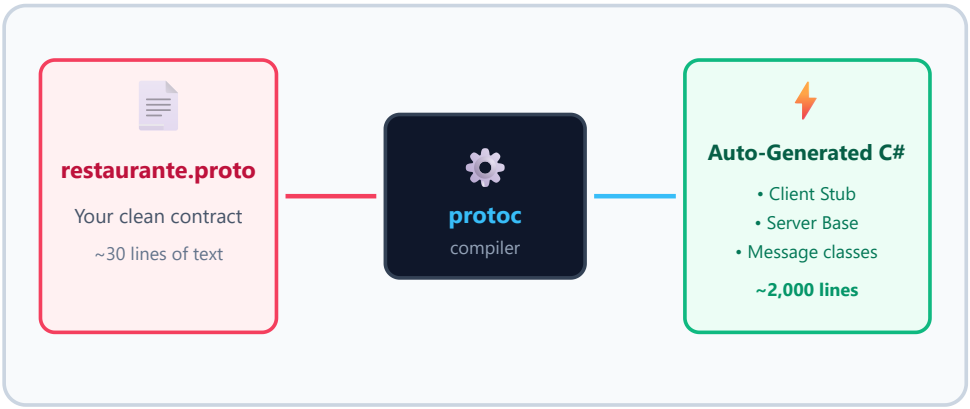
These are **Unary RPCs**: the client sends one request and receives one response.

> 📝 **Notes: Original Lab Dots (...)**
> The lab left field types and tags as placeholder dots `...`. In Step 1.5, we give you the exact, complete types for every field!

Code Generation (protoc)

Why we never write network serialization by hand

Writing network sockets and binary encoders manually takes weeks. The **protoc compiler** generates all code automatically:

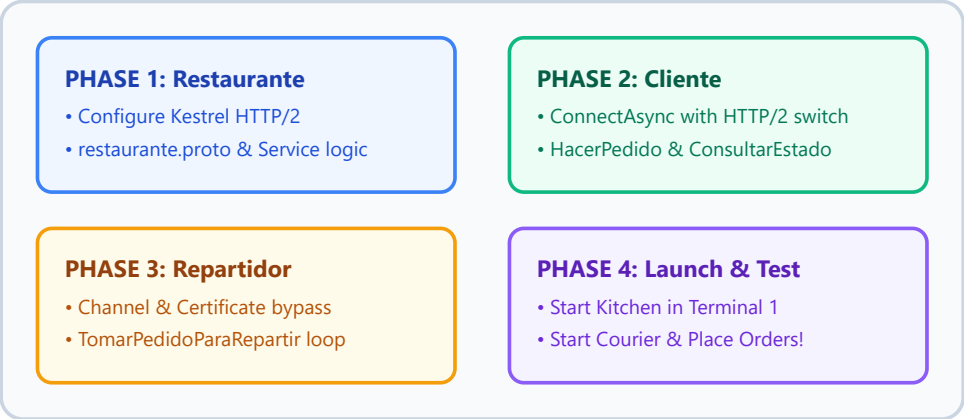


> 📌 **Remember: Never edit generated code!**
> Whenever you run `dotnet build`, `Grpc.Tools` regenerates these files. Edit `restaurante.proto` instead!

Lab Roadmap

Your step-by-step path from zero to complete

We will implement the lab across four clear phases:



Notes: Project Templates

The lab uses starter folders from Campus Virtual. If building from scratch, run `dotnet new console` and add the `Grpc.AspNetCore` NuGet package.

LAB STEP 1.1 & 1.2 · SERVER SETUP

Configuring Kestrel

Enabling HTTP/2 without TLS on port 5000

Goal: Configure Kestrel to listen on HTTP/2 without TLS on port 5000 and enable gRPC support.

What to do: In `Restaurante/Program.cs`, add before `builder.Build()`:

```
// Step 1.1: Listen on port 5000 using HTTP/2 without TLS
builder.WebHost.ConfigureKestrel(options => {
    options.ListenLocalhost(5000, o => o.Protocols = HttpProtocols.Htt
});

// Step 1.2: Register gRPC middleware in ASP.NET Core
builder.Services.AddGrpc();
```

Why we do it: Standard web traffic uses HTTP/1.1. gRPC requires HTTP/2. This setting enables plain HTTP/2 for local lab development.

Expected result: Server configures Kestrel for HTTP/2 on port 5000.

> ⚠️ **Warning: Missing Http2 protocol setting**
> If you omit `o.Protocols = HttpProtocols.Http2`, Kestrel uses HTTP/1.1 and all gRPC calls fail with status code 426!

LAB STEP 1.3 & 1.4 · SERVICE REGISTRATION

Singleton & Endpoint Mapping

Sharing one kitchen queue across all incoming requests

Goal: Register `RestauranteService` as a Singleton and map gRPC routes to it.

What to do: In `Restaurante/Program.cs`:

```
// Step 1.3: Register service as Singleton to preserve state
builder.Services.AddSingleton<RestauranteService>();

var app = builder.Build();

// Step 1.4: Map gRPC endpoints and fallback route
app.MapGrpcService<RestauranteService>();
app.MapGet("/", () => "Servidor gRPC del Restaurante ejecutándose...");
```

Why we do it: A **Singleton** creates only one service instance. All orders stay in the same memory queue!

Expected result: Server maps incoming RPC calls to our singleton service.

> 💡 **Why Singleton instead of Transient?**
> If registered as `Transient`, ASP.NET creates a new service instance for every call, and orders vanish instantly!

LAB STEP 1.5 & 1.6 · PROTO CONTRACT

Writing restaurante.proto

Completing the contract with exact types and field tags

Goal: Define all messages and RPC methods in `restaurante.proto`.

What to do: In `Restaurante/Protos/restaurante.proto`, add:

```
syntax = "proto3";
import "google/protobuf/empty.proto";
option csharp_namespace = "Restaurante";

message PlatoPedido { string nombre = 1; int32 cantidad = 2; }
message PedidoRequest { repeated PlatoPedido platos = 1; int32 distanc
message PedidoResponse { int32 id_pedido = 1; string mensaje = 2; }
message ConsultaRequest { int32 id_pedido = 1; }
message EstadoResponse { string estado = 1; string mensaje = 2; int32
```

Why we do it: Gives protoc the blueprint to generate type-safe C# classes.

Expected result: `dotnet build` compiles without errors.

> 📝 **Notes: Missing Import in Lab**
>
> The lab uses `google.protobuf.Empty` for couriers, but omits `import "google/protobuf/empty.proto";`. Always include this import at the top!

LAB STEP 1.7 · STATE & SYNCHRONIZATION

Concurrency & Shared State

Protecting the order queue from race conditions

Goal: Declare and initialize the shared queue, lock object, and semaphore.

What to do: In `Restaurante/Services/RestauranteService.cs`:

```
private readonly ConcurrentQueue<Pedido> _ordersQueue = new();
private const int MaxQueueCapacity = 10;
private readonly SemaphoreSlim _readyOrdersSemaphore = new(0);
private readonly object _takeLock = new();
private int _orderID = 1;
```

Why we do it: Customers and couriers call the kitchen simultaneously. `SemaphoreSlim` signals sleeping couriers when food is ready. `_takeLock` prevents race conditions.

Expected result: Thread-safe variables ready in the kitchen.

> 📝 **Notes: Semaphore Initial Count = 0**
>
> Always initialize `_readyOrdersSemaphore` to `0`! Couriers must sleep until the first order is placed.

LAB STEP 1.8 · KITCHEN LOGIC

Implementing HacerPedido

Safely storing orders and ringing the courier bell

Goal: Accept customer orders safely and wake up waiting couriers.

What to do: In `RestauranteService.cs`:

```
public override Task<PedidoResponse> HacerPedido(PedidoRequest req, Se
  lock (_takeLock) {
    if (_ordersQueue.Count < MaxQueueCapacity) {
      var p = new Pedido(_orderID, req.Platos.ToList(), req.Distancia)
      _ordersQueue.Enqueue(p);
      _orderID++;
      _readyOrdersSemaphore.Release(); // Ring bell: wake 1 courier!
      return Task.FromResult(new PedidoResponse {
        IdPedido = p.Id, Mensaje = "Pedido en preparación"
      });
    }
  }
  throw new RpcException(new Status(StatusCode.ResourceExhausted, "Col
}
```

Why we do it: `lock` prevents ID conflicts. `Release()` increments the semaphore to awaken one waiting delivery driver.

Expected result: Returns new Order ID and wakes a driver.

> 📌 **Remember: The Semaphore Bell**
>
> `_readyOrdersSemaphore.Release()` is the chef ringing the counter bell. One waiting courier wakes up instantly!

LAB STEP 1.9 & 1.10 · STATUS & DELIVERY

Status Check & Pickup

Tracking food and dispatching couriers

Goal: Allow customers to track orders and couriers to claim ready food.

What to do: In `RestauranteService.cs`:

```
public override Task<EstadoResponse> ConsultarEstadoPedido(ConsultaReq
  var p = _ordersQueue.FirstOrDefault(o => o.Id == req.IdPedido);
  if (p == null) throw new RpcException(new Status(StatusCode.NotFound
  return Task.FromResult(new EstadoResponse { Estado = p.Estado, Tiemp
}

public override async Task<PedidoResponse> TomarPedidoParaRepartir(Emp
  await _readyOrdersSemaphore.WaitAsync(); // Sleep until food is read
  lock (_takeLock) {
    if (_ordersQueue.TryDequeue(out var p)) {
      p.Estado = "En reparto";
      return new PedidoResponse { IdPedido = p.Id, Mensaje = "Pedido t
    }
  }
  throw new RpcException(new Status(StatusCode.NotFound, "Sin pedidos
}
```

Why we do it: `WaitAsync()` suspends the courier call asynchronously with zero CPU waste until an order is ready.

Expected result: Couriers take orders in strict FIFO order.

> 💡 **Why WaitAsync() instead of a while loop?**
> A `while(true)` loop burns 100% CPU polling. `WaitAsync()` pauses the thread at zero cost!

Connecting the Customer

Creating the channel and client stub

Goal: Initialize the gRPC channel and create the client stub.

What to do: In `Cliente/Services/RestauranteClientService.cs`:

```
public async Task ConnectAsync() {
  if (_serverAddress.StartsWith("http://", StringComparison.OrdinalIgn
    AppContext.SetSwitch("System.Net.Http.SocketsHttpHandler.Http2Unen
  }
  _channel = GrpcChannel.ForAddress(_serverAddress, new GrpcChannelOpt
  _client = new RestauranteService.RestauranteServiceClient(_channel);
  Console.WriteLine($"Conectado al servidor en {_serverAddress}");
}

public async Task DisconnectAsync() {
  if (_channel != null) { await _channel.ShutdownAsync(); }
}
```

Why we do it: .NET blocks unencrypted HTTP/2 for security. `SetSwitch` unlocks it for local labs.

Expected result: Prints `"Conectado al servidor en http://localhost:5000"`.

> 💡 **What is RestauranteServiceClient?**
> It is the auto-generated **Client Stub** (the waiter). It exposes C# methods that package requests and transmit them over HTTP/2.

LAB STEP 2.4 & 2.5 · CLIENTE OPERATIONS

Placing & Tracking Orders

Invoking remote methods and handling errors

Goal: Implement `HacerPedidoAsync` and `ConsultarEstadoPedidoAsync` in the client.

What to do: In `RestauranteClientService.cs`:

```
public async Task<PedidoResponse> HacerPedidoAsync(List<PlatoPedido> p
  var req = new PedidoRequest { Distancia = dist };
  req.Platos.AddRange(platos);
  try {
    var res = await _client.HacerPedidoAsync(req);
    Console.WriteLine($"Pedido #{res.IdPedido}: {res.Mensaje}");
    return res;
  } catch (RpcException ex) { Console.WriteLine($"Error: {ex.Status.De
}

public async Task<EstadoResponse> ConsultarEstadoPedidoAsync(int id) {
  try {
    var res = await _client.ConsultarEstadoPedidoAsync(new ConsultaReq
    Console.WriteLine($"Estado: {res.Estado}, Tiempo: {res.TiempoEstim
    return res;
  } catch (RpcException ex) { Console.WriteLine($"Error: {ex.Status.De
}
```

Why we do it: Sends typed requests over the network and catches remote exceptions.

Expected result: Customer can place orders and query cooking time.

> 📌 **Remember: Repeated Protobuf Fields**
>
> Protobuf `repeated` lists are read-only collections in C#. Populate them using `.AddRange()` instead of direct assignment!

The Courier Worker

Waiting patiently for orders to deliver

Goal: Connect the courier app and wait for available orders.

What to do: In `Repartidor/Program.cs`:

```
var httpHandler = new HttpClientHandler();
httpHandler.ServerCertificateCustomValidationCallback =
    HttpClientHandler.DangerousAcceptAnyServerCertificateValidator;

var channel = GrpcChannel.ForAddress("http://localhost:5000",
    new GrpcChannelOptions { HttpHandler = httpHandler });
var client = new RestauranteService.RestauranteServiceClient(channel);

// Claim an order
try {
  Console.WriteLine("Esperando pedidos disponibles...");
  var res = await client.TomarPedidoParaRepartirAsync(new Empty());
  Console.WriteLine($"¡Pedido #{res.IdPedido} asignado para entrega!")
} catch (RpcException ex) { Console.WriteLine($"Error: {ex.Status.Deta
```

Why we do it: Courier calls the server and sleeps until the kitchen rings the bell.

Expected result: When a customer orders, the courier instantly receives it.

> 📌 **Remember: Multiple Couriers Scale Safely**
>
> If multiple couriers run simultaneously, each order is claimed by exactly one courier thanks to `SemaphoreSlim` and `lock`!

LAB STEP 4 · SYSTEM LAUNCH

Launching the System

Watching all three distributed programs collaborate

Goal: Run all three applications across separate terminals and watch them interact.

What to do: Open three terminal tabs in VS Code or PowerShell:

```
# Terminal 1: Start the Kitchen Server first
dotnet run --project Restaurante

# Terminal 2: Start the Delivery Courier (waits patiently)
dotnet run --project Repartidor

# Terminal 3: Start the Customer and place an order!
dotnet run --project Cliente
```

Why we do it: Verifies real end-to-end communication across 3 distinct processes over HTTP/2.

Expected result: Placing an order in Terminal 3 immediately wakes up Terminal 2 to deliver it!

> 📌 **Remember: Launch Order Matters!**
>
> Always start the **Restaurante server first** so that clients and couriers have an active server to connect to.

Common Mistakes & Fixes

How to diagnose and solve the top four beginner traps

If something does not work as expected, check these four common pitfalls:

Trap 1: Orders disappear immediately

Cause: Registered service as `AddScoped` or `AddTransient`.
Fix: Use `builder.Services.AddSingleton<RestauranteService>()`.

Trap 2: HTTP 426 or "Bad HTTP/1.1 Request"

Cause: Kestrel defaulted to HTTP/1.1 without TLS.
Fix: Set `o.Protocols = HttpProtocols.Http2` and set `Http2UnencryptedSupport` switch to `true`.

Trap 3: Courier claims empty order right away

Cause: `SemaphoreSlim` was initialized with count 1 instead of 0.
Fix: Initialize with `new SemaphoreSlim(0)` so couriers sleep until food is ready.

Trap 4: protoc fails with "Empty is not defined"

Cause: Missing protobuf standard import.
Fix: Add `import "google/protobuf/empty.proto";` at top of `restaurante.proto`.

SECTION 9 · QUICK REFERENCE

1-Page Cheat Sheet

The entire gRPC distributed lab at a single glance

gRPC Concept	Restaurant Analogy	What it does in C#
Client	Hungry Customer	Console app calling `_client.HacerPedidoAsync()`
Server	The Kitchen	ASP.NET Core host listening on port 5000
.proto file	The Menu Contract	Interface definition used by `protoc`
Client Stub	The Waiter	Auto-generated class serializing calls into bytes
HTTP/2	Order Tube	High-speed binary transport with multiplexing

Tool / Method	Purpose & Golden Rule
`AddSingleton`	Keeps one shared queue instance in server memory.
`lock (_takeLock)`	Ensures exclusive access to check capacity and increment IDs.
`SemaphoreSlim(0)`	Couriers sleep with `WaitAsync()` until kitchen calls `Release()`.
`ConcurrentQueue<T>`	Thread-safe FIFO queue for incoming customer orders.

📌 **Golden Rule**

Never edit generated code by hand. Modify `restaurante.proto` and let `dotnet build` do the generation!

SECTION 9 · KNOWLEDGE CHECK

5 Quiz Questions

Test and prove your understanding of gRPC

Q1: What do numbers like `= 1;` in a .proto file represent?

Ans: Unique binary field tags on the wire, not default values!

Q2: Why must `RestauranteService` be a Singleton?

Ans: So all clients and couriers share the same order queue in memory.

Q3: Why use `SemaphoreSlim` instead of a while loop?

Ans: A while loop burns 100% CPU. `WaitAsync()` sleeps with 0% CPU.

Q4: What is the Client Stub's main job?

Ans: It serializes objects into binary and hides network complexity.

Q5: How does gRPC differ from REST/JSON?

Ans: gRPC uses compact binary Protobuf over HTTP/2 instead of text JSON.

🎉 **Congratulations!**

You have mastered distributed systems with gRPC!